

## Sprawozdanie sk2

Sieciowa implementacja gry logicznej Rummikub w architekturze klient-serwer

### 1. Opis projektu

Celem projektu było stworzenie sieciowej wersji gry liczbowej Rummikub, umożliwiającej rozgrywkę wieloosobową (od 2 do 4 graczy) w czasie rzeczywistym. System oparty jest na architekturze klient-serwer, gdzie serwer zarządza logiką, stanem gry i walidacją ruchów, a klient odpowiada za interfejs graficzny i interakcję z użytkownikiem.

Użyte technologie:

- Serwer: Język C++14, uruchamiany w środowisku GNU/Linux. Komunikacja sieciowa oparta na API Berkeley Sockets (sys/socket.h). Do obsługi wielu połączeń jednocześnie wykorzystano mechanizm multipleksacji I/O przy użyciu funkcji systemowej poll(). Zastosowano gniazda nieblokujące (O\_NONBLOCK).
- Klient: Język Python 3 z biblioteką pygame do renderowania grafiki oraz modułem socket i threading do obsługi komunikacji sieciowej.
- Protokół: Własny protokół tekstowy, w którym komunikaty rozdzielane są znakiem nowej linii (\n).

### 2. Opis komunikacji pomiędzy serwerem i klientem

Komunikacja odbywa się za pośrednictwem protokołu TCP, co gwarantuje kolejność i dostarczenie pakietów, co jest kluczowe w turowej grze strategicznej.

Przebieg komunikacji:

1. Nawiązanie połączenia: Klient łączy się z serwerem i wysyła komendę JOIN <tryb> <nick> (np. JOIN 2 User1). Serwer dodaje gracza do odpowiedniej kolejki oczekujących.
2. Start gry: Po zebraniu odpowiedniej liczby graczy serwer tworzy instancję klasy Room, rozdaje klocki i rozsyła GAME\_START oraz WELCOME\_HAND z początkowym układem ręki.
3. Tura gry:
  - Synchronizacja wizualna: Gdy gracz przesuwa klocek, klient wysyła pakiety POS <id> <x> <y>. Serwer rozsyła je do pozostałych (broadcast), aby widzieli ruch w czasie rzeczywistym, jednak nie zatwierdza jeszcze tych zmian jako ostatecznych.
  - Zatwierdzenie ruchu: ZGracz kończy turę wysyłając COMMIT <układ\_grup> lub dobiera klocek (DRAW).
  - Walidacja: Serwer parsuje otrzymane grupy i uruchamia GameValidator. Jeśli ruch jest poprawny (zgodność sum, kolorów, ciągłości), serwer aktualizuje stan stołu. W przypadku błędu (np. rozbita grupa, suma < 30 przy otwarciu), serwer wykonuje przywrócenie stanu sprzed tury (rollback) i nakłada karę (dobranie klocka).

4. Obsługa błędów: W przypadku rozłączenia gracza, serwer wykrywa zdarzenie (flagi POLLHUP/POLLERR w pętli poll), kończy grę w danym pokoju i informuje pozostałych graczy komunikatem GAME\_OVER.

### 3. Podsumowanie

Implementacja serwera jest jednowątkowa i oparta na zdarzeniach (event-driven), co uzyskano dzięki funkcji poll(). Pozwala to na wydajną obsługę wielu pokoi gier jednocześnie bez konieczności tworzenia osobnego wątku dla każdego klienta.

Do obsługi ruchów gracza zastosowano podejście transakcyjne: współrzędne klocków są aktualizowane na bieżąco ("wizualnie"), ale logiczna struktura stołu zmienia się dopiero po komendzie COMMIT. W przypadku wykrycia błędu przez walidator, serwer wykonuje natychmiastowy powrót do stanu zapisanego w backup\_table i backup\_current\_hand. Gwarantuje to spójność gry nawet przy próbach oszustwa lub błędach klienta.

Opracowano własny, tekstowy protokół aplikacyjny. Komunikaty są atomowe i kończą się znakiem nowej linii. Dane o strukturze planszy są serializowane do formatu łańcuchowego (np. POS id x y color value), co ułatwia debugowanie i parsowanie.

Aplikacja kliencka w Pythonie wykorzystuje dwa wątki. Główny wątek odpowiada za pętlę renderowania pygame i obsługę zdarzeń UI. Drugi wątek (NetworkClient) nasłuchuje w tle na pakiety z serwera. Synchronizacja między wątkami odbywa się za pomocą bezpiecznej wątkowo kolejki queue.Queue, co eliminuje problem "zamrażania" okna gry podczas oczekiwania na dane z sieci.

Trudności i wyzwania:

- Synchronizacja stanu wizualnego i logicznego: Największą trudnością było rozwiązanie problemu interaktywnego przesuwania klocków. Klient wizualizuje ruchy natychmiastowo, wysyłając pakiety POS, ale serwer nie może ich od razu zatwierdzać logicznie. Wymagało to rozróżnienia na serwerze stanu "tymczasowego" (współrzędne) i "zatwierdzonego" (własność i grupy). Wprowadzenie mechanizmu backupu (backup\_table) rozwiązało problem, umożliwiając cofnięcie błędnych manipulacji gracza.
- Walidacja zasad Rummikuba: Wyzwaniem było zaimplementowanie poprawnego algorytmu walidacji po stronie serwera (GameValidator::validateBoard), który musi uwzględniać manipulację Jokerem oraz specyficzne zasady "otwarcia" (zakaz ingerencji w stół i minimum 30 punktów z ręki).
- UI Klienta (Insert): Mechanizm wstawiania klocka pomiędzy inne ("rozsuwanie") okazał się trudny do synchronizacji w trybie multiplayer. Ostatecznie został zaimplementowany jedynie dla obszaru "ręki" gracza, gdzie zmiany są lokalne i nie wymagają ciągłej synchronizacji pozycji z innymi graczami.